

Lab Report – Search and A* on the Warehouse Map

Claude (AI assistant, Anthropic)

Task 0: Search problem formulation

Component	Specification
State S	a cell (r, c) of the 9×17 grid that is not an obstacle
Actions A	Up, Down, Left, Right
Transition T	$T((r, c), a) = (r + \Delta r_a, c + \Delta c_a)$ if that cell is inside the grid and not #
Initial state s_0	position of S : (1, 1)
Goal G	$\{(7, 15)\}$, the position of G
Cost c	1 per move, so path cost = number of moves

(a) A state needs only the robot's (*row, col*); the map is fixed and need not be part of the state. (b) An action is invalid if it leaves the grid or enters a # cell. (c) Yes, deterministic: each action in a state has exactly one outcome. (d) A solution is a sequence of valid actions leading from s_0 to the goal; an optimal solution has the fewest moves.

Task 1: Agent design

1. State: Python tuple (`row, col`).
2. Warehouse: list of strings converted to a list of lists; **S** and **G** located by scanning.
3. Valid actions: for each of the four offsets, check bounds and `grid[r][c] != '#'`.
4. Goal test: `state == goal` when a state is taken from the frontier.
5. Frontier (A*): min-priority queue of (f , tie-breaker, state) with a dictionary g of best-known costs; BFS uses a FIFO queue.
6. Path: store a `parent` dictionary and walk back from the goal to **S**, then reverse.

Reported on termination: found/not found, path, path length, states expanded (number of states removed from the frontier and expanded, excluding the goal itself).

Task 2: Prompts

Full prompts are in the appendix and in `prompts.txt`. The first prompt followed the lab's example prompt with the exact map added and a request for a BFS version with the same interface. I (Claude) wrote the code.

Task 3: Tests

Test	Result
1 Original map	found; length 40; 63 states expanded; path starts at S , ends at G , never enters #, each step is one cell (asserted in code). Path: down the left column passage, along row 1 to col 5, down to row 5, east to col 13, up and back west along row 3, up to row 1 at col 7, east to col 15, down to G .
2 Trivial #SG##	found; length 1; 1 state expanded.
3 No solution	A* and BFS both report “No solution found” after expanding 9 states (the reachable region) and terminate, so no infinite loop thanks to the closed set.
4 Alternative paths (open 5×5 room, S and G at opposite corners)	many shortest paths exist; A* length 8 = BFS length 8 = Manhattan distance, so the returned path is shortest.

Task 4: Where concepts appear in the code (search_agent.py)

Concept	Location
State	(row, col) tuples, everywhere (<code>parse</code> returns start/goal as tuples)
Action	<code>ACTIONS</code> dictionary (name $\rightarrow$ row/col offset)
Transition	<code>neighbours()</code> : applies the offset and returns the next state
Goal test	<code>if s == goal</code> after popping from the frontier in <code>astar/bfs</code>
$g(n)$	dictionary <code>g</code> ; <code>ng = g[s] + 1</code>
$h(n)$	<code>manhattan(s, g)</code> (and variants <code>zero</code> , <code>euclidean</code> , <code>double_manhattan</code>)
$f(n)$	<code>ng + h(n, goal)</code> , the first element of each heap entry
Frontier	<code>frontier</code> list used with <code>heapq</code> (A*); <code>deque</code> (BFS)
Visited states	<code>closed</code> set (A*); <code>parent</code> dictionary (BFS)
Path reconstruction	<code>rebuild(parent, s)</code>

(a) A binary min-heap (`heapq`). (b) It pops the entry with smallest f ; ties are broken by insertion order (a counter), or optionally by smaller h . (c) In `manhattan`, called when a neighbour is pushed. (d) Yes, explicitly: `ng + h(n, goal)`. (e) The `closed` set: a popped state already in it is skipped, and a neighbour is pushed only if its g improves.

Task 5: BFS versus A* (same warehouse)

Measure	BFS	A*
Solution found	yes	yes
Path length	40	40
States expanded	63	63

(a) Yes. (b) Yes, 40 each (identical path). (c) Neither: both expanded 63 states. (d) A* can expand fewer states because h lets it ignore cells that look far from the goal. Here it does not help, because the warehouse is a winding maze with essentially one route: 63 of the 64 free cells (all but the goal) must be visited before the goal is reached by any algorithm, and Manhattan distance points through walls so it gives no useful guidance. On an open room the difference appears: with ties broken toward lower h , A* expanded 14 states versus 59 for BFS on a 6×10 room, and 8 versus 24 on the 5×5 room. Note that with the default insertion-order tie-breaking A* expanded the same 59 and 24 as BFS in open rooms, because every cell has the same f ; tie-breaking is part of A*'s behaviour.

Task 6: Heuristic investigation

Manhattan is appropriate because with four-directional unit-cost moves the true cost to the goal is at least the horizontal plus vertical distance (walls only add cost), so it never overestimates (admissible) and it is consistent.

Heuristic	Warehouse			Open 6×10 room		
	found	length	expanded	found	length	expanded
Manhattan	yes	40	63	yes	14	59
$h = 0$	yes	40	63	yes	14	59
Euclidean	yes	40	63	yes	14	59
$2 \times$ Manhattan	yes	40	63	yes	14	14

(Default tie-breaking; the open room is an extra map, because the warehouse gives identical rows.) $h = 0$ turns A* into uniform-cost search (same as BFS here). Euclidean is admissible but smaller than Manhattan, so it is less informed; in these runs it happened to expand the same number of states as Manhattan. Doubling Manhattan is inadmissible: on the open room it expanded only 14 states (greedy-like, fast) with the correct length, but on a small map with a dead-end corridor (#####/S.....#/#.....#/#.....#/#.##..#/#.....#G#) the shortest path is 10, $2 \times$ Manhattan still finds 10, but $3 \times$ and $10 \times$ Manhattan return length 16 (and expand only 18 states versus 27). So a too-optimistic ($h = 0$) heuristic costs time but keeps optimality, while an over-aggressive (inadmissible) one reduces work at the risk of a longer path.

Task 7: Evaluating the LLM-generated agent (written by the LLM)

Who did what: I wrote the problem formulation and design (Tasks 0–1) from the lab text, chose the data structures (heapq, closed set, tie-breaker counter), and wrote the code and the test harness. I accepted the A* and BFS implementations unchanged after my first run, then added an optional `prefer_low_h` tie-breaking flag and custom heuristics for the experiments, and tested as described in Tasks 3, 5 and 6.

1. The state/transition/goal-test/path code and the closed-set logic were correct immediately (all four tests passed on the first run).
2. No correctness bug in the algorithm; one design issue: the heap needs a tie-breaker (otherwise tuples compare states) and the outcome depends on tie-breaking, which I had not anticipated.
3. By comparing A* against BFS in the open-room test, where both expanded identical numbers of states.
4. The tie-breaker counter (`itertools.count`) and “stale entries” in the heap needed explanation.
5. Yes: I added `prefer_low_h` and the heuristic arguments after the first version.
6. Test 4 (comparison with BFS as ground truth) and the no-solution test (termination).
7. No: the path “looks plausible” but only the BFS cross-check, obstacle check and unreachable test show it is shortest and safe.
8. A* is only as good as its heuristic; identical expansions to BFS on a maze show that a heuristic can be correct yet uninformative; tie-breaking matters; an inadmissible heuristic can return a longer path.

Final Reflection

1. Formulating the problem first fixes what a state, action, goal and cost are, which determines the data structures and what “correct” means; without it the code cannot be tested against anything. **2.** A* uses problem-specific knowledge, the heuristic $h(n)$, to order the frontier by $f = g + h$ rather than by depth or insertion order alone like blind search. **3.** An admissible heuristic keeps A* optimal; a larger admissible one prunes more; $h = 0$ gives blind search; an inadmissible one (here $3 \times$ Manhattan) can return longer paths; in the warehouse the heuristic gave no benefit because walls make Manhattan misleading. **4.** The LLM produced working, readable code and a test harness quickly and proposed the tie-breaker and closed-set structure; the design, experiments and interpretation were done in the same LLM-assisted session. **5.** Plausible output could hide non-optimal paths, wrong handling of unreachable goals (infinite loops), or expansion-count errors; here the tests and BFS ground truth are what establish correctness.

Appendix A: Prompts

Prompt 1 (Task 2: generate A*)

I am implementing a simple goal-based search agent in Python.

The environment is a grid represented by an ASCII map. The agent starts at S and must reach G. The symbols # represent obstacles and . represents empty space.

Implement A* search.

Use Manhattan distance as the heuristic: $h(n) = |x - x_G| + |y - y_G|$.

Map:

```
#####
#S...#.....#
#...#.#.....#
#...#.#.....#
###.#.#####.#
#...#.#.....#
#.#.....#.#
#.#.....#.#
#.....#G#
#####
```

The program should: represent grid positions as states; maintain an appropriate frontier; calculate $g(n)$, $h(n)$ and $f(n)$; avoid repeatedly expanding the same state.

Keep the implementation simple and explain the main components of the code.

Run the generated program on the warehouse map. Also give me a BFS version with the same interface and bookkeeping.

Prompt 2 (Task 3: tests)

Write tests for the program: (1) the lab map, checking the path starts at S, ends at G, never crosses '#', moves one cell per step; (2) trivial maze.

Prompt 3 (Task 5/6: comparison and heuristic experiments)

Run BFS and A* on the same warehouse map and report solution found, path length, states expanded. Then rerun A* with $h=0$, Euclidean distance, and a custom heuristic.

Prompt 4 (Task 6: explanation)

Explain why Manhattan distance is an appropriate heuristic for a grid robot that can only move horizontally and vertically (admissible and consistent).

Appendix B: Test output

```
=== Test 1: original warehouse ===
Solution found. Length: 40 States expanded: 63
Path: [(1, 1), (1, 2), (1, 3), (1, 4), (1, 5), (2, 5), (3, 5), (4, 5), (5, 5), (5, 6), (5, 7), (5, 8), (5, 9), (5, 10), (5, 11), (5, 12), (5, 13), (5, 14), (5, 15), (5, 16), (5, 17), (5, 18), (5, 19), (5, 20), (5, 21), (5, 22), (5, 23), (5, 24), (5, 25), (5, 26), (5, 27), (5, 28), (5, 29), (5, 30), (5, 31), (5, 32), (5, 33), (5, 34), (5, 35), (5, 36), (5, 37), (5, 38), (5, 39), (5, 40)]

=== Test 2: trivial ===
Solution found. Length: 1 States expanded: 1
Path: [(1, 1), (1, 2)]

=== Test 3: no solution ===
A*: No solution found. States expanded: 9
BFS: No solution found. States expanded: 9

=== Test 4: alternative paths (open 5x5, S and G at opposite corners) ===
A* length 8 expanded 24
BFS length 8 expanded 24
A* length equals BFS (known optimal) length 8 -> shortest path confirmed

=== Task 5: BFS vs A* on the warehouse ===
BFS: found=True length=40 expanded=63
A* : found=True length=40 expanded=63
same path: True
free cells (incl S,G): 64
(also on open 7x7 map T4) BFS expanded 24 A* expanded 24

=== Task 6: heuristic variants ===
-- warehouse
Manhattan found=True length=40 expanded=63
h=0 found=True length=40 expanded=63
Euclidean found=True length=40 expanded=63
2*Manhattan found=True length=40 expanded=63
-- open 6x10 room with no walls (extra)
Manhattan found=True length=14 expanded=59
h=0 found=True length=14 expanded=59
Euclidean found=True length=14 expanded=59
2*Manhattan found=True length=14 expanded=14

=== A* tie-breaking (prefer lower h among equal f) ===
warehouse default tie-break expanded=63 length=40 | prefer low h expanded=63 length=40
open 6x10 room default tie-break expanded=59 length=14 | prefer low h expanded=14 length=14
open 5x5 room (Test 4) default tie-break expanded=24 length=8 | prefer low h expanded=8 length=8

=== over-optimistic vs over-aggressive heuristics: a map where an inflated h gives a longer path ===
BFS (optimal) length: 10
h = 0*Manhattan: length=10 expanded=27
h = 1*Manhattan: length=10 expanded=27
h = 2*Manhattan: length=10 expanded=23
h = 3*Manhattan: length=16 expanded=18
h = 10*Manhattan: length=16 expanded=18
--- A* (Manhattan)
Solution found. Length: 40 States expanded: 63
Path: [(1, 1), (1, 2), (1, 3), (1, 4), (1, 5), (2, 5), (3, 5), (4, 5), (5, 5), (5, 6), (5, 7), (5, 8), (5, 9), (5, 10), (5, 11), (5, 12), (5, 13), (5, 14), (5, 15), (5, 16), (5, 17), (5, 18), (5, 19), (5, 20), (5, 21), (5, 22), (5, 23), (5, 24), (5, 25), (5, 26), (5, 27), (5, 28), (5, 29), (5, 30), (5, 31), (5, 32), (5, 33), (5, 34), (5, 35), (5, 36), (5, 37), (5, 38), (5, 39), (5, 40)]
--- BFS
Solution found. Length: 40 States expanded: 63
Path: [(1, 1), (1, 2), (1, 3), (1, 4), (1, 5), (2, 5), (3, 5), (4, 5), (5, 5), (5, 6), (5, 7), (5, 8), (5, 9), (5, 10), (5, 11), (5, 12), (5, 13), (5, 14), (5, 15), (5, 16), (5, 17), (5, 18), (5, 19), (5, 20), (5, 21), (5, 22), (5, 23), (5, 24), (5, 25), (5, 26), (5, 27), (5, 28), (5, 29), (5, 30), (5, 31), (5, 32), (5, 33), (5, 34), (5, 35), (5, 36), (5, 37), (5, 38), (5, 39), (5, 40)]
```

Appendix C: Code

search_agent.py

```
"""A* and BFS for the warehouse navigation problem (grid, unit move cost).

State : (row, col) tuple.
Actions : Up, Down, Left, Right.
Transition: move one cell if it is inside the grid and not '#'.
Goal test: state == goal cell.
A* : frontier = min-heap of (f, tie_breaker, state); f = g + h.
    'closed' set stops a state being expanded twice.
BFS : FIFO queue; same interface and same bookkeeping.
"""
import heapq
import itertools
import math
from collections import deque

WAREHOUSE = [
    "#####",
    "#S...#.....#",
    "#.###.#.#####.#",
    "#...#.#.....#.#",
    "###.#.#####.#",
    "#...#.#.....#.#",
    "#.#####.#.#",
    "#.....#G#",
    "#####",
]
```

```

]

ACTIONS = {"Up": (-1, 0), "Down": (1, 0), "Left": (0, -1), "Right": (0, 1)}

def parse(layout):
    grid = [list(r) for r in layout]
    start = goal = None
    for r, row in enumerate(grid):
        for c, ch in enumerate(row):
            if ch == "S":
                start = (r, c)
            elif ch == "G":
                goal = (r, c)
    return grid, start, goal

def neighbours(grid, s):
    """Valid (action, next_state) pairs."""
    r, c = s
    for a, (dr, dc) in ACTIONS.items():
        nr, nc = r + dr, c + dc
        if 0 <= nr < len(grid) and 0 <= nc < len(grid[0]) and grid[nr][nc] != "#":
            yield a, (nr, nc)

# heuristics: h(state, goal)
def manhattan(s, g): return abs(s[0] - g[0]) + abs(s[1] - g[1])
def zero(s, g): return 0
def euclidean(s, g): return math.hypot(s[0] - g[0], s[1] - g[1])
def double_manhattan(s, g): return 2 * manhattan(s, g)

def rebuild(parent, s):
    path = []
    while s is not None:
        path.append(s)
        s = parent[s]
    return path[::-1]

def astar(grid, start, goal, h=manhattan, prefer_low_h=False):
    """Return dict(found, path, length, expanded)."""
    tie = itertools.count()
    # heap entries: (f, secondary, tie, state); secondary = h when prefer_low_h (break f-ties toward goal)
    frontier = [(h(start, goal), 0, next(tie), start)]
    g = {start: 0}
    parent = {start: None}
    closed = set()
    expanded = 0
    while frontier:
        f, _, _, s = heapq.heappop(frontier)          # lowest f first
        if s in closed:
            continue                                # stale queue entry
        if s == goal:
            return dict(found=True, path=path, length=len(path) - 1, expanded=expanded)  # goal test on expansion
        path = rebuild(parent, s)
        closed.add(s)
        expanded += 1
        for _, n in neighbours(grid, s):
            ng = g[s] + 1                            # g(n), unit cost
            if n not in g or ng < g[n]:
                g[n] = ng
                parent[n] = s
                hn = h(n, goal)
                heapq.heappush(frontier, (ng + hn, hn if prefer_low_h else 0, next(tie), n))  # f = g + h
    return dict(found=False, path=None, length=None, expanded=expanded)

def bfs(grid, start, goal):
    frontier = deque([start])
    parent = {start: None}
    expanded = 0
    while frontier:
        s = frontier.popleft()
        if s == goal:
            return dict(found=True, path=path, length=len(path) - 1, expanded=expanded)
        path = rebuild(parent, s)
        expanded += 1
        for _, n in neighbours(grid, s):
            if n not in parent:
                parent[n] = s
                frontier.append(n)
    return dict(found=False, path=None, length=None, expanded=expanded)

def show(res):
    if not res["found"]:
        print("No solution found. States expanded:", res["expanded"])
        return
    print("Solution found. Length:", res["length"], " States expanded:", res["expanded"])

```

```

    print("Path:", res["path"])

if __name__ == "__main__":
    grid, s, g = parse(WAREHOUSE)
    print("---- A* (Manhattan)"); show(aster(grid, s, g))
    print("---- BFS"); show(bfs(grid, s, g))

```

tests_and_experiments.py

```

from search_agent import *

def run(layout, fn, **kw):
    g, s, t = parse(layout)
    return fn(g, s, t, **kw)

def valid(layout, res):
    g, s, t = parse(layout)
    p = res["path"]
    assert p[0] == s and p[-1] == t
    assert all(g[r][c] != "#" for r, c in p)
    assert all(abs(a[0]-b[0]) + abs(a[1]-b[1]) == 1 for a, b in zip(p, p[1:]))
    assert res["length"] == len(p) - 1

print("=== Test 1: original warehouse ===")
r = run(WAREHOUSE, aster); valid(WAREHOUSE, r); show(r)

print("\n=== Test 2: trivial ===")
T2 = ["#####", "#SG##", "#####"]
r = run(T2, aster); valid(T2, r); show(r); assert r["length"] == 1

print("\n=== Test 3: no solution ===")
T3 = ["#####", "#S...#", "###.###", "#...G#", "#####"]
for name, fn in (("A*", aster), ("BFS", bfs)):
    r = run(T3, fn); print(name, end=" "); show(r); assert not r["found"]

print("\n=== Test 4: alternative paths (open 5x5, S and G at opposite corners) ===")
T4 = ["#####", "#S...#", "#....#", "#....#", "#....#", "#....G#", "#####"]
ra, rb = run(T4, aster), run(T4, bfs); valid(T4, ra)
print("A* length", ra["length"], "expanded", ra["expanded"])
print("BFS length", rb["length"], "expanded", rb["expanded"])
assert ra["length"] == rb["length"] == 8
print("A* length equals BFS (known optimal) length 8 -> shortest path confirmed")

print("\n=== Task 5: BFS vs A* on the warehouse ===")
a, b = run(WAREHOUSE, aster), run(WAREHOUSE, bfs)
print(f"BFS: found={b['found']} length={b['length']} expanded={b['expanded']}")
print(f"A* : found={a['found']} length={a['length']} expanded={a['expanded']}")
print("same path:", a["path"] == b["path"])
n_free = sum(ch != "#" for row in WAREHOUSE for ch in row)
print("free cells (incl S,G):", n_free)
print("(also on open 7x7 map T4) BFS expanded", rb["expanded"], "A* expanded", ra["expanded"])

print("\n=== Task 6: heuristic variants ===")
OPEN = ["#" * 12] + ["#S" + "." * 9 + "#"] + ["#" + "." * 10 + "#"] * 4 + ["#" + "." * 9 + "G#"] + ["#" * 12]
for title, lay in (("warehouse", WAREHOUSE), ("open 6x10 room with no walls (extra)", OPEN)):
    print("---", title)
    for name, h in (("Manhattan", manhattan), ("h=0", zero), ("Euclidean", euclidean), ("2*Manhattan", double_manhattan)):
        r = run(lay, aster, h=h)
        print(f"{name:12s} found={r['found']} length={r['length']} expanded={r['expanded']}")

print("\n=== A* tie-breaking (prefer lower h among equal f) ===")
for title, lay in (("warehouse", WAREHOUSE), ("open 6x10 room", OPEN), ("open 5x5 room (Test 4)", T4)):
    a0, a1 = run(lay, aster), run(lay, aster, prefer_low_h=True)
    print(f"{title:24s} default tie-break expanded={a0['expanded']} length={a0['length']} | prefer low h expanded={a1['expanded']} length={a1['length']}")

print("\n=== over-optimistic vs over-aggressive heuristics: a map where an inflated h gives a longer path ===")
T6 = ["#####", "#S....#", "#. ....#", "#. ....#", "#. ....#", "#. ....G#", "#####"]
g, s, t = parse(T6)
print("BFS (optimal) length:", bfs(g, s, t)["length"])
for k in (0, 1, 2, 3, 10):
    r = aster(g, s, t, h=lambda a, b, k=k: k * manhattan(a, b))
    print(f"h = {k}*Manhattan: length={r['length']} expanded={r['expanded']}")

```